Architectural Isolation as a Timing Safety Primitive for Edge AI Medical Devices: Controlled Experimental Evidence on a Shared-Silicon Platform

Akul Mallayya Swami

***Abstract*—A system can satisfy accuracy-based validation, maintain output stability ($\mathrm{STER}{=}0.0000$), and still violate timing constraints under deployment load—these are structurally independent properties that current pre-market validation does not distinguish. This letter demonstrates that these properties are independent through a controlled same-hardware experiment: identical MobileNetV2 models are evaluated under identical adversarial load on two execution paths of the same NVIDIA Jetson Orin Nano Super—a dedicated GPU accelerator (TensorRT FP16) and a general-purpose CPU (ONNX Runtime FP32). Across five stressor classes and $\approx 158{,}000$ inference activations, both paths maintain $\mathrm{STER}{=}0.0000$, confirming output instability is not the dominant failure mode. The paths diverge sharply on timing: the GPU path maintains latency <15 ms under all conditions; the CPU path degrades $7.2\times$ under combined load (14.5 ms to 104.0 ms mean, $\mathrm{P99}{=}165.1$ ms), breaching the 10 Hz clinical cycle budget by 65%. The Safety-Threshold Exceedance Rate (STER) and latency verification are proposed jointly as a candidate pre-market acceptance protocol under FDA Draft Guidance FDA-2024-D-4488 (January 2025).**

***Index Terms*—edge AI, inference stability, timing safety, architectural isolation, Safety-Threshold Exceedance Rate (STER), Software as a Medical Device (SaMD), pre-market validation, IEC 62304, ISO 14971, TensorRT, ONNX Runtime.**

I. Introduction

FDA pre-market validation of AI-enabled medical devices evaluates model accuracy (sensitivity, specificity, AUC) under controlled, zero-contention conditions [1]. This framework does not explicitly characterize whether inference outputs remain within bounds, or whether inference timing is preserved within clinical cycle budgets, when the device operates under realistic deployment load. A 2025 cross-sectional study of 950 FDA-cleared AI-enabled devices found that diagnostic errors were the leading cause of AI device recalls, with 43% occurring within the first year of clearance [15]. IEC 62304 Clause 6.3.2 requires verification that software meets its timing requirements under all foreseeable operating conditions [8]; however, to our knowledge, no standardized empirical method exists for characterizing whether inference pipeline timing is preserved under realistic deployment stress. Current pre-market validation frameworks can certify a system as functionally correct while it simultaneously violates real-time operational constraints under deployment load—a failure mode that is structurally invisible to accuracy-based testing.

This letter is not a performance optimization study; it exposes a validation failure: standard accuracy-based certification cannot detect operational constraint violations that occur under deployment load. The novelty is not timing degradation—that is known—but the validation blindness to it. The safety-relevant threshold is not latency magnitude but latency relative to the operational deadline at a fixed clinical activation rate. This letter addresses this gap through a controlled experiment in which all hardware variables are held constant, so any observed divergence arises from execution context—isolating the software execution path as the operative factor. This letter demonstrates that accuracy-based validation can certify systems that violate timing constraints under deployment load, and introduces STER to isolate output stability from temporal safety—two properties current validation does not distinguish. This work does not study timing degradation itself; it exposes a validation failure where such degradation remains undetected under accuracy-based certification ($\Delta\mathrm{acc}{<}1.0\%$ on both paths). The controlled same-hardware experiment shows STER = 0.0000 on both paths while CPU-path latency degrades $7.2\times$ to P99 = 165.1 ms under combined load; a joint STER + latency pre-market protocol operationalizing FDA-2024-D-4488 is proposed.

II. Related Work

Prior edge inference work addresses throughput maximization [3], latency predictability under GPU partitioning [4], and cross-device accuracy benchmarking [5], [6]. These works measure latency *or* output accuracy, not their independence under shared-silicon contention. Safety-oriented work on neural networks in medical devices [7] addresses model-level mechanisms but does not empirically characterize resource-contention-induced timing failure as a distinct safety property from output correctness. Prior work optimizes or characterizes inference performance and isolation within GPU execution; this letter demonstrates a validation-level failure that is independent of performance optimization—the contribution is problem class separation, not methodological novelty alone [4].

III. System Model and Formal Definitions

A. Output Stability Under Identical Inputs

Let f denote a deployed inference model and $\mathbf{x}_i$ a fixed test input ($i = 1, \ldots, N$). Under a stressor condition c,

A. M. Swami is with Varian Medical Systems, A Siemens Healthineers Company, Palo Alto, CA, USA (e-mail: swami.akul@alumni.uml.edu). ORCID: 0009-0003-9549-5543. Portions of this manuscript were drafted with the assistance of Claude (Anthropic). The author reviewed, verified, and takes full responsibility for all content, experimental data, and claims.

let $\sigma(\mathbf{y}_i^c)$ denote the softmax output and $\bar{\sigma}_i$ the per-image reference softmax captured at zero load. The per-inference output deviation is:

$$\delta_i = \left\|\sigma(\mathbf{y}_i^c) - \bar{\sigma}_i\right\|_\infty = \max_k \left|\sigma(\mathbf{y}_i^c)_k - \bar{\sigma}_{i,k}\right| \quad (1)$$

B. STER Definition

Definition 1 (T^*): $T^* = 0.05$, selected as conservative relative to ISO 14971 Table B.1 clinical diagnostic tolerance bands (typically 10–20%).

Definition 2 (STER): For N inference activations:

$$\mathrm{STER} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\delta_i > T^*] \quad (2)$$

Accuracy testing evaluates $\mathrm{argmax}(\sigma(\mathbf{y}))$—a single class label—and cannot detect bounded distribution drift below the argmax threshold; this is a structural limitation, not a sensitivity issue. A model whose top-1 prediction is unchanged but whose confidence distribution has shifted by 4.9% passes accuracy testing. STER captures this class of silent deviations; accuracy cannot by construction. STER is proposed as a candidate pre-market metric, not an existing FDA-specified standard. This threshold reflects conservative tolerance bounds relative to ISO 14971 Table B.1 risk control examples, which typically reference 10–20% functional deviation thresholds.

C. Robustness Confirmation Condition

The joint pre-market acceptance condition is:

$$\mathrm{STER}_c \leq \mathrm{STER}_{\max} \quad \text{AND} \quad \mathrm{latency}_{\mathrm{P99}} \leq T_n \quad (3)$$

where $\mathrm{STER}_{\max} = 0$ and $T_n = 100\,\mathrm{ms}$ (10 Hz cycle budget). 10 Hz is a representative real-time clinical monitoring constraint, not a universal SaMD requirement; appropriate activation rates are device- and application-specific. STER and latency are orthogonal safety dimensions: a device may satisfy (3) fully (GPU path, E0–E5), satisfy only the STER term (CPU path, E6), or fail both. Both must be verified. A timing violation is defined as deviation beyond a threshold that compromises deadline adherence at a fixed clinical activation rate, where missed deadlines preclude synchronous data delivery to downstream decision processes. The CPU-path P99 of 165.1 ms constitutes such a violation at 10 Hz: 1% of inferences miss the cycle budget by $\geq$65%, and this violation persists undetected under accuracy-based validation (Δacc$<$1.0%). Such deviations are incompatible with systems requiring bounded execution intervals; they can propagate to downstream decision pipelines, invalidating synchronization assumptions and compromising system-level reliability.

D. Architectural Model

The Jetson Orin Nano Super uses shared LPDDR5: CPU, GPU, and memory controller compete for the same bus. The *GPU execution path* (TensorRT FP16) offloads inference to the 1024-core Ampere GPU via dedicated Direct Memory Access (DMA) paths with Quality of Service (QoS) bandwidth reservation, isolating inference from CPU scheduling pressure.

TABLE I
HARDWARE PLATFORM SUMMARY

Component	Role	Key Specification
Jetson Orin Nano Super 8GB (GPU path)	Primary inference host	1024-core Ampere GPU, TensorRT FP16, shared LPDDR5
Jetson Orin Nano Super 8GB (CPU path)	Negative control (same hardware)	ONNX Runtime FP32, CPUExecutionProvider, Cortex-A78AE
nRF52840 DK + TI SensorTag	BLE network stressor	BLE 5.0, up to 6 simultaneous connections
GL-AX1800 WiFi 6 Router	RF environment	2.4 GHz co-channel interference

The *CPU execution path* (ONNX Runtime FP32) runs on the 6-core Arm Cortex-A78AE cluster, directly competing with stressor workloads for CPU time, cache, and DRAM bandwidth. This architectural difference is hypothesized to produce divergent timing behavior under contention while preserving output correctness on both paths; this is empirically tested in E1–E6.

IV. Hardware Platform and Experimental Protocol

A. Hardware Platform

Both execution paths run MobileNetV2 [12] (224×224 RGB, ImageNet 1000-class head) on the same 500-image fixed test set, held constant across all experiments. The GPU path runs TensorRT FP16 [13] at a nominal 10 Hz ($T_n = 100\,\mathrm{ms}$); the CPU path runs MobileNetV2 FP32 via ONNX Runtime [14] on the same hardware. Using the same physical device for both paths is the critical methodological choice: it eliminates chip-to-chip variation and thermal differences as confounds, supporting attribution of timing behavior to the accelerator architecture. Jetson surface temperature was monitored via tegrastats throughout all trials; no thermal throttling events were recorded and sustained clock frequencies were confirmed across all conditions.

B. Experimental Protocol

For each trial, $N = 500$ inference activations are recorded. For each activation i: (1) fixed test input $\mathbf{x}_i$ is presented; (2) softmax $\sigma(\mathbf{y}_i)$ is recorded; (3) δ_i is computed per (1) against the per-image zero-load reference; (4) latency is recorded via `time.perf_counter()`. STER is computed per (2) over the N activations. Top-1 accuracy is computed as a control variable. All CPU inferences used CPUExecutionProvider in deterministic mode with a fixed thread count and no graph optimizations that introduce nondeterminism. Trial counts (N=500 per condition, 10 trials per stressor level) were selected to provide stable mean and P99 latency estimates; the 7.2× divergence represents a practically significant difference independent of formal hypothesis testing.

TABLE II
EXPERIMENT SUMMARY

ID	Stressor	Protocol	IEC
E0	Baseline	Both paths, zero load. Per-image reference $\bar{\sigma}_i$ captured. 10 trials × 500 inferences.	B+C
E1	CPU stress	stress-ng at 25/50/75/100% system-wide load. Both paths tested at each level. 10 trials × 500 inferences per level.	B
E2–E4	Isolation stressors	Memory pressure (25–90% fill), GPU co-tenancy (0–4 workers), network I/O (BLE 0–6 conns + WiFi). GPU path only; STER = 0.0000 across all conditions. 5–10 trials per level.	B/C
E5	Combined load	CPU 75% + memory 50% + BLE 4 conns + fio disk write. Both paths. 10 trials × 500 inferences. *Worst-case deployment scenario.*	C
E6	CPU negative control	CPU path only (no GPU). Conditions matching E0 + E1 + E5. 10 trials × 500 inferences per condition. Primary contrastive result.	B

TABLE III
E0 BASELINE: BOTH EXECUTION PATHS

Path	STER	δ_{mean}	Lat. mean	Lat. P99
GPU (TensorRT FP16)	0.0000	0.0181	14.8 ms	≈18 ms
CPU (ONNX FP32)	0.0000	0.0000	14.5 ms	18.6 ms

TABLE IV
E1 CPU STRESS: BOTH PATHS (10 TRIALS × 500 INFERENCES PER LEVEL)

CPU Load	GPU Path		CPU Path	
	STER	Lat. mean	STER	Lat. mean
25%	0.0000	≈14.8 ms	0.0000	33.4 ms
50%	0.0000	≈14.8 ms	0.0000	57.8 ms
75%	0.0000	≈14.8 ms	0.0000	73.7 ms
100%	0.0000	≈14.8 ms	0.0000	70.9 ms

V. RESULTS

A. E0: Baseline Calibration

Both paths establish matched zero-load baselines (Δlatency <0.3 ms), confirming controlled experimental conditions. GPU floating-point variance produces $\delta_{\text{mean}}=0.0181$ but zero exceedances above $T^*=0.05$; CPU ONNX Runtime is arithmetically deterministic ($\delta=0.0000$ exactly). Top-1 accuracy: 1.6% on both paths (same model, same images, same label mapping; accuracy values reflect ImageNet label remapping and are not used as performance benchmarks).

B. E1: CPU Stress and Timing Divergence

Both paths maintain STER = 0.0000 at all CPU load levels; timing diverges significantly: the GPU path is invariant (≈14.8 ms regardless of load) while the CPU path reaches 73.7 ms at 75% load (5.1× baseline). TensorRT FP16 executes on the GPU, isolated from saturated CPU cores; ONNX Runtime FP32 competes for CPU time slices and cache bandwidth, degrading latency without affecting arithmetic output.

TABLE V
E6 CPU-ONLY NEGATIVE CONTROL: JETSON ORIN NANO SUPER, ONNX RUNTIME FP32 (10 TRIALS × 500 INFERENCES PER CONDITION)

Condition	STER	δ_{mean}	Lat. mean	Lat. P99
Zero load	0.0000	0.0000	14.5 ms	18.6 ms
CPU 25%	0.0000	0.0000	33.4 ms	111.4 ms
CPU 50%	0.0000	0.0000	57.8 ms	115.8 ms
CPU 75%	0.0000	0.0000	73.7 ms	122.1 ms
CPU 100%	0.0000	0.0000	70.9 ms	117.0 ms
Combined	**0.0000**	**0.0000**	**104.0 ms**	**165.1 ms**†

† Exceeds T_n=100 ms cycle budget by 65%.

C. E2–E4: Isolation Stressors (GPU Path)

Memory pressure (25–90% LPDDR5 fill), GPU co-tenancy (0–4 concurrent TensorRT workers), and network I/O (BLE 0–6 connections with WiFi co-channel interference) all produce STER = 0.0000 with δ_{mean} indistinguishable from E0 baseline on the GPU path ($\delta_{\text{mean}}=0.0181\pm0.0002$ across all conditions, $n>25{,}000$ inferences per stressor class). GPU DMA isolation, QoS bandwidth reservation, and independent TensorRT contexts prevent all stressor classes from perturbing outputs or latency; a confirmed +88% interrupt rate at BLE conns = 4 had no effect.

D. E5: Combined Realistic Load

Under simultaneous CPU 75% + memory 50% fill + BLE 4 connections + fio disk write, the GPU path maintains STER = 0.0000 and latency <15 ms (mean 14.9 ms, $\delta_{\text{mean}}=0.0156\pm0.0001$) across all 10 trials. The CPU path maintains STER = 0.0000 but reaches 104.0 ms mean / 165.1 ms P99, exceeding the 10 Hz cycle budget by 65%.

E. E6: CPU Negative Control

STER = 0.0000 and $\delta=0.0000$ exactly across all 50,000 inferences: ONNX Runtime's CPUExecutionProvider uses deterministic ARM NEON SIMD operations; scheduling pressure delays computation without perturbing its arithmetic. Latency degrades monotonically with stressor intensity, reaching 7.2× baseline under combined load. P99 = 165.1 ms under combined load means 1% of inferences exceed the clinical cycle budget by ≥65%. The GPU path, under identical combined load, maintains latency <15 ms across all 10 trials. E6 establishes that the dedicated GPU accelerator's contribution to the safety profile is *timing isolation*, not numerical stability; this is a property of deterministic floating-point execution regardless of accelerator.

The observed timing divergence is not itself the contribution; the contribution is that this divergence remains undetected under standard accuracy-based validation despite violating operational constraints—a system passes certification while being undeployable at the required activation rate.

VI. Discussion

The contribution is not the presence of timing variation under load—that is expected system behavior—but the structural inability of existing accuracy-based validation to detect it, which this controlled experiment demonstrates empirically.

A. Controlled Design as Methodological Contribution

The same-hardware design is the critical methodological choice. These results constitute engineering-level evidence for a safety-relevant validation primitive, not a complete clinical protocol; further validation against regulated device workflows is required before 510(k) adoption. Running GPU and CPU paths on the same Jetson Orin Nano Super controls for all hardware-level confounds; timing divergence of $7.2\times$ under identical stressors is consistent with a causal interpretation attributing divergence to the accelerator architecture, as cross-platform comparisons would leave hardware differences as alternative explanations. Platform datasheets document architectural separation in principle; this work provides empirical quantification at clinical deployment intensities.

B. Timing as the Dominant Failure Mode

STER = 0.0000 on both paths establishes that output instability is not the failure mode; the dominant failure mode is timing degradation, invisible to STER, accuracy, and any output-only evaluation method. This motivates timing characterization as a complementary pre-market verification property.

C. Pre-Market Validation Gap

A manufacturer using accuracy-only testing would clear both execution paths as equivalent; only the joint condition (3) distinguishes them, as $\Delta\mathrm{acc}<1.0\%$ on both paths while the CPU-only path breaches the 10 Hz cycle budget by 65%.

D. STER as Output Stability Certification

STER certifies the full output distribution $\sigma(\mathbf{y})$ within T^*—a property accuracy ($\mathrm{argmax}(\sigma(\mathbf{y}))$) structurally cannot confirm—thereby isolating timing as the sole safety-relevant divergence under all tested conditions.

E. Limitations

Results are specific to MobileNetV2 at 10 Hz on Jetson Orin Nano Super under JetPack 6; models with higher memory bandwidth utilization or non-deterministic preprocessing may exhibit different behavior. $T^*=0.05$ is conservative relative to ISO 14971 Table B.1 but is not clinically validated for any specific device class. The GPU and CPU paths differ in execution hardware and runtime stack (TensorRT vs. ONNX Runtime); future work isolating a single runtime would further strengthen the attribution claim. This study characterizes inference-layer timing and output stability only and does not evaluate preprocessing latency, real-time OS scheduling, multi-model pipelines, or a complete regulated clinical workflow.

VII. Conclusion

This letter has demonstrated, through a controlled same-hardware experiment, that output correctness and timing reliability are independent safety properties of edge AI inference pipelines. Running MobileNetV2 under identical adversarial stressors on the GPU and CPU execution paths of the same Jetson Orin Nano Super, both paths maintain STER = 0.0000 across $\approx 158{,}000$ inference activations; the GPU path maintains latency <15 ms while the CPU path degrades $7.2\times$ to 104.0 ms mean / 165.1 ms P99 under combined load, breaching the 10 Hz clinical cycle budget. Current accuracy-based FDA pre-market testing would pass both paths without detecting this timing failure. STER provides a mechanism to verify output distribution bounds; latency verification under contention is required for timing safety. Output correctness alone is insufficient for reliable clinical deployment; temporal behavior under deployment contention must be explicitly validated. Joint STER + latency verification is proposed, operationalizing FDA-2024-D-4488 and consistent with IEC 62304 and ISO 14971.

References

[1] FDA, "Artificial Intelligence-Enabled Device Software Functions: Lifecycle Management and Marketing Submission Recommendations," Draft Guidance, Docket FDA-2024-D-4488, Jan. 2025.

[2] FDA, "Good Machine Learning Practice for Medical Device Development: Guiding Principles," Oct. 2021.

[3] J. Hao, P. Subedi, L. Ramaswamy, and I. K. Kim, "Reaching for the Sky: Maximizing Deep Learning Inference Throughput on Edge Devices with AI Multi-Tenancy," *ACM Trans. Internet Technol.*, vol. 23, no. 1, pp. 1–33, Feb. 2023.

[4] J. J. Martín, A. Castillo, J. García, and F. J. Cazorla, "Performance Isolation for Inference Processes in Edge GPU Systems," arXiv:2601.07600, Jan. 2026.

[5] S. P. Baller, A. Jindal, M. Chadha, and M. Gerndt, "DeepEdgeBench: Benchmarking Deep Neural Networks on Edge Devices," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, 2021, pp. 20–30.

[6] R. Tobiasz *et al.*, "Edge Devices Inference Performance Comparison," arXiv:2306.12093, Jun. 2023.

[7] U. Becker, "Increasing Safety of Neural Networks in Medical Devices," in *Proc. SAFECOMP Workshops*, LNCS vol. 11699, Springer, 2019, pp. 91–101.

[8] IEC 62304:2006+AMD1:2015, "Medical device software: Software life cycle processes," IEC, Geneva, 2015.

[9] ISO 14971:2019, "Medical devices: Application of risk management to medical devices," ISO, Geneva, 2019.

[10] C. I. King, "stress-ng: A Tool to Load and Stress a Computer System," GitHub, 2023. [Online]. Available: https://github.com/ColinIanKing/stress-ng

[11] NVIDIA Corporation, "Holoscan SDK: A Platform for Medical Device Developers," NVIDIA Developer Documentation, 2024. [Online]. Available: https://developer.nvidia.com/holoscan-sdk

[12] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520.

[13] NVIDIA Corporation, "TensorRT Developer Guide," NVIDIA Developer Documentation, 2024. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/

[14] Microsoft Corporation, "ONNX Runtime: Cross-Platform Inference Accelerator," GitHub, 2024. [Online]. Available: https://github.com/microsoft/onnxruntime

[15] B. Lee *et al.*, "Early Recalls and Clinical Validation Gaps in Artificial Intelligence-Enabled Medical Devices," *JAMA Health Forum*, vol. 6, no. 8, p. e253172, Aug. 2025.